

System testing optimization in a globally distributed software engineering team

Tulasi Anand, Chittoor Reddy
Siemens Healthcare Pvt. Ltd.
Bangalore, India

tulasi.anand@siemens.com; chittoor.reddy@siemens.com

V. S. Mani
Siemens Technology and Services Pvt. Ltd.
Bangalore, India
vs.mani@siemens.com

Abstract—this practice paper presents how a test team of a software engineering organization spread across three countries successfully optimized the system testing effort for a large mission-critical software system that had to conform to regulatory requirements. Multiple releases of the system of the system have been delivered to customers over the 15 years it has been in the market. Each new release added new features, which increased both the number of system test cases required and the complexity of the existing system test cases. Hence, a complete cycle of system testing for every new release required more time and effort. However, it was increasingly difficult to justify or arrange for such added efforts, especially since shorter product releases were being demanded. This created a vicious cycle where the team members, though experienced and highly-skilled, were unable to focus on initiatives to improve and optimize testing. To address this issue, we introduced the role of a test analyst to bridge the gap between the test team and other key stakeholders. We were able to optimize the total number of system test cases and increase test execution efficiency while ensuring conformance to mandatory regulations. The practices adapted by the team for faster test environment preparation, assisted automation, along with training and mentoring are described.

Keywords—test management, test strategy, test refactoring, stake holder involvement, agile, global software engineering, regulatory requirements.

I. INTRODUCTION

This practice paper presents the experience of a test team of a globally distributed software engineering organization in reducing the system testing effort for a large system, without modifying the test specifications significantly or resorting to additional test automation or demanding excessive effort from testers and other stake holders. Agile development methodology is used [1], [2].

II. THE SETUP OF THE GLOBAL TEAM

The software engineering team was spread across four locations in three countries. The distributed team worked on systems that enable users to manage confidential customer information along with acquisition, storage and management of images. These systems had to conform to stringent regulatory requirements. The test teams accounted for about 20 per cent of overall engineering team.

Each location works on different parts of the overall system and has a team of test engineers who have about four to twelve years experience in the domain. The test engineers collaborate to achieve location-specific test goals, which are derived from

the overall test objectives. The scope of test team is integration and system testing.

A key deliverable of the test team is the test specifications document, which specifies test cases that usually test related functionalities. The test case specifications are typically unstructured and derived from relevant workflows based on the domain expertise of the test engineers who were at different geographic locations (see figure 1).

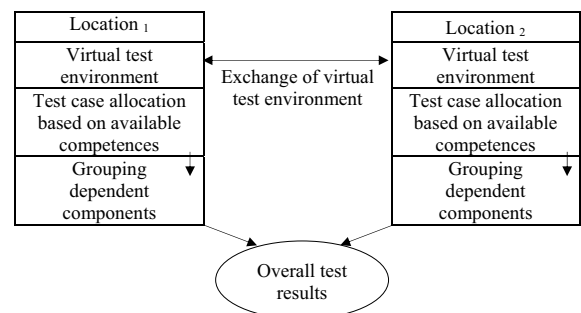


Figure 1: Schematic of test approach

III. CHALLENGES

This section lists the challenges in optimizing system testing.

Legacy: The system under test included several legacy sub-systems that were not adequately testable. This ruled out use of extensive test automation to achieve the objective of test sufficiency, resulting in increased manual testing. Also, since multiple releases of the system had been delivered over the 15 years it had been in the market, the suite of test specifications had increased significantly, needing additional manual testing effort.

Interdependence: Each location tested sub-systems developed in that location (see figure1). The aim of the system tests is to verify the functionality and validate that the sub-systems conform to requirements. Due to the dependencies between the sub-systems, functionalities in other sub-systems that were out of scope of testing for a location were typically covered as well, resulting in duplication of test effort.

Target environments: The overall system functionality had to be tested on different target environments that mirror actual deployments in customer locations. Hence, it was important to

identify the required target environments in addition to optimizing the system testing suite.

Limited benefits from applying test design techniques: Though test design techniques [3] are can potentially optimize the number of test cases, they could not be directly used. This is because the rigorous test review process to assure the test cases adequately fulfill their objectives requires time and effort. It was not feasible to recreate the reviewed legacy test cases since the review effort exceeded the gains in optimization.

Progressively increasing number and complexity of test cases: Every new product release added new features, which had a double impact on the test cases. One, new features increased the total number of test cases. Two, new features also increased the complexity of the existing test cases that were incrementally modified. Hence, a complete cycle of testing of the application required progressively more time. In addition, in many instances it took less effort to create a new test case than to find and modify a suitable existing test case from the large number of test cases.

Test environment preparation: The test environment preparation, which included installation, configuration, and test data preparation; was not only complex but also consumed a large amount of time. It accounted for about 10 percent of total system test effort. The procedure to prepare test environment was manual and error prone which often resulted in unproductive effort.

IV. MANAGEMENT DIRECTIVE

It was increasingly difficult to justify or arrange for increased test effort, given the growing demand for shorter product releases to the end customers.

After identifying the challenges, the test team defined an approach to reduce the system testing effort and presented it to the management. The management agreed with the need to reduce the overall test effort and set a goal of 20 percent effort reduction.

V. APPROACH

The approach involved addressing the challenges by evaluating all possibilities of reducing test effort while working within quality processes and resource constraints - mainly time and effort. This excluded any significantly effort-intensive methods, such as rewriting all the test cases or large scale automation. As part of the approach we implemented several practices that are given below. The practices also served the purpose of bridging the gap between several stake holders of the project [4].

A. Detecting redundancies and uncovering potential for optimization

We analyzed test specification documents that had a likely overlap of functionality tested with the aim of detecting redundancies and uncovering potential for optimization.

In this phase, the feasibility to automate a test case was also assessed. In addition, other process improvements that could reduce the test effort, like faster installation cycle, usage of tool

that provides easier editing of test cases, test data preparation time were considered.

B. Identifying test specifications suitable for optimization

While considering optimization, it had to be ensured that there was no loss of functionality covered. This meant that any valid test step should not be bypassed as a result of the optimization.

Criteria for selecting a test specification for optimization was an optimization potential of 10 percent. Optimization potential referred to the reduction in test cases possible without affecting functionality covered. Selected test specifications were taken up for modification.

C. Identifying other improvements

Though test automation was to a large extent infeasible, we had automated bring up tests (test cases that verify the most basic functionality of the application). The test automation framework created for bring-up tests had a potential to automate test steps included in a few test specifications. Certain changes were proposed in these test specifications, to automate test steps that consumed a disproportionately large share of test execution effort. Automating these test steps assisted manual test execution, thus reducing the overall effort.

VI. PRACTICES

A. Technical approach

1) Optimizing when there are multiple test cases per requirement

It was observed that if a test specification had two or more test cases for a requirement they could contain duplicate test steps. By selecting and analyzing such test cases, redundant steps between the test cases were identified and removed. However the guidance was not considered, if the requirement was large enough to justify two or more test cases. Product owners helped test engineers assess test cases that were to be merged or removed. This practice reduced total system test effort by about 6 percent.

2) Same requirement traced to two or more test specifications

Similar functionality was getting tested more than once under certain conditions. This was also due to the fact that different testers from different locations who authored those test specification in different releases were unaware of the overlap (see table 1). We found it feasible to remove such test cases, which were already available in another test specification. It was also possible to merge such test cases, and test the functionality fully. Identifying and removing or merging such overlapping test cases reduced the total system testing effort by about 1 percent.

3) Repeated test case in different test configurations

Traditionally, some test cases were getting re-executed in all application configurations. While it was necessary to execute some test cases in all application configurations, it was not required for many test cases, as it did not increase system test coverage. Adequacy of testing in a subset of test configuration per test case was decided only after a discussion with developer and the architect. The count of test cases did

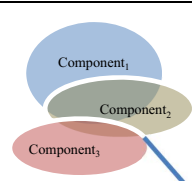
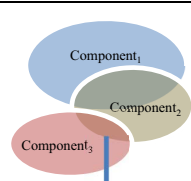
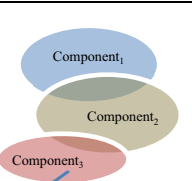
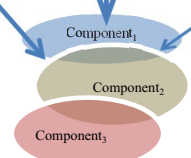
not decrease in this case. However the unnecessary repeated execution of same test cases was avoided, which shortened the test execution time and reduced total system test effort by 2 percent. This also helped in reducing the gap in the understanding between architects/developers and testers.

4) Merging two levels of system testing

The software requirements were divided into problem level and solution level requirements. The problem level requirements were those that state the intended functionality as a 'problem statement', with a description as a use case. The solution level requirements provide more implementation and application user interface details. The test cases that trace to problem level requirement keys have to cover the workflow level user actions while the solution level test cases have to cover more detailed application level interactions.

As a consequence, 2 levels of test cases were written. There were instances, where both the requirements were described in a similar way or the functionality did not mandate a more detailed test case. In such cases, related test cases for each of the levels also contained similar test steps. When such test cases were analyzed together redundancies were detected. Redundant test cases were merged after an assessment by product owners and testers. This has reduced total system test effort by about 2 percent.

TABLE I. INCREASING OVERLAP BETWEEN TWO LOCATIONS/SPECIFICATIONS OVER A PERIOD OF TIME

Location 1	Location 2	Location 3
Uses Software Requirement Specification ₁ (SRS ₁)	Uses Software Requirement Specification ₂ (SRS ₂)	Uses Software Requirement Specification ₃ (SRS ₃)
Tests Component1 as part of SRS ₁	Tests Component2 as part of SRS ₂	Tests Component3 as part of SRS ₃
Also needs to test parts of Component ₂ and Component ₃ , as part of workflow tests to satisfy requirements of SRS ₁	Also needs to test parts of Component ₁ and Component ₃ , as part of workflow tests to satisfy requirements of SRS ₂	Also needs to test parts of Component ₁ and Component ₂ , as part of workflow tests to satisfy requirements of SRS ₃
Overlapped parts of the applications are tested as part of 3 documents and locations	Overlapped parts of the applications are tested as part of 3 documents and locations	Overlapped parts of the applications are tested as part of 3 documents and locations
		
Overlapped test cases/test steps between each of the components are merged at one test document/location instead of getting tested 3 times		Overlapped test cases/test steps between each of the components are merged at one test document/location instead of getting tested 3 times

5) Interface functionality tested at two levels

The system level testing for use case level requirement keys is intended for overall workflow testing. However, it is not appropriate to test lower level interfaces in system tests, but they need to be tested in the integration test level. The discussions held with Product owners and Testers has enhanced the understanding of appropriateness of a test level. By avoiding repeated interface system tests it was possibility to reduce total system test effort by about 1 percent.

6) Automation

The application already had a test automation framework. An evaluation was done to identify the test cases that were suitable for automation and use only the existing framework. A group of test cases were identified which had the support from test automation framework already or required minor modifications. This approach facilitated a rapid development of test automation suite at system test level, with little effort. On the whole, automation reduced the total system test effort by about 3 percent.

7) Assisted installation and configuration

Manual installation of the application was effort intensive. The process of installation and configuration was a series of complex steps, taking up to 0.5 person days per installation. Since testers typically need to install the system about 3-4 times for one complete system test cycle, automating system installation offered promise. In addition, automation had an added benefit of minimizing errors during installation, which required time consuming re-installation. Test automation engineers worked with manual testers to provide required guidelines to automate system installation. By using automation scripts to install the system reduced the total system test effort by about 2 percent.

8) Reducing the time taken for bring-up tests

The bring-up tests were executed as a batch of test cases, which required changes to the configuration files after a test case is executed and before the next test case starts executing. After the configuration changes, several application programs/services need to be restarted for the changes to take effect. Previously, for every configuration file change, the entire set of application programs/services were being restarted, which consumed a considerable amount of time. Testers working with architects proposed approaches to optimize the time taken, by identifying and restarting only the necessary application programs/services. This required no changes to the existing test environment and only minor changes to the existing test scripts. Restarting only the necessary application programs/services, between two test cases, has shortened the time to execute the bring-up tests and has reduced the total system test effort by about 1 percent.

9) Focus on high impact test specifications

To make test specification optimization effective and to be in synch with other project goals, a prioritization of the test specifications taken up for update based on knowledge and impact of changes was done. This approach was similar to data mining methodology of [5], but with a focus on specification with highest impact and intended change. This also helped in meeting the overall *done* criterion.

10) Other methods

Some of the other measures adapted are listed below. They have reduced the total the system test effort by about 1 percent.

- As part of an organizational initiative, training and deploying a new test specification editing tool that reduced the time to create, modify, and review test specifications. Frequent updates to test specifications were reduced and were done only when necessary, in agreement with test management.
- By including a second tester as peer reviewer during the creation of test specification, created a back up author. This facilitated easier transition of authors, when needed.
- By providing training on test case design, it was possible to create simpler and more uniform test cases faster.
- Feature based and smaller SCRUM teams have enhanced the collaboration and enhanced the knowledge sharing, hand holding and hence reduced time for learning.
- Movement of identified requirements to an earlier test phase, as described in [6], reduced the time taken for system testing.
- Selecting regression test cases based on impact reduced the total test cases selected for execution. The distribution of test cases to different sites was guided by competences available at the sites. Virtualized test environment and sharing of virtual 'images' of different application across multiple sites, helped eliminate tester-specific differences in installation or configuration (see Figure 1). Common database and tools for defect tracking helped greatly in communicating the defects to development teams in different locations, who had to make corrections.

B. Planning and execution

1) Sprint/iteration level planning

The plan involving entire team was prepared, resulting in better test team coordination through the stand-up meetings covering different locations.

Once the effort estimate for implementation of the identified modifications was available, the activity was integrated in to the sprint/iteration level planning. The back log - containing the list of test specifications being updated, the author, effort, reviewer, and likely date of review - helped planning the tasks. The *done* criterion, of successfully releasing the documents after optimization of test cases was defined. This enabled assessing the degree of completion of planned tasks for the sprint.

C. Communication and training

1) Communication

It is important not only to set a goal and make plan for implementation, but also to communicate the plan to the test team and other stakeholders. The test managers from different location meet three times a week, using conferencing tools to

exchange data and information regarding the test planning, progress, and support required from other stakeholders. Subsequently, they communicate essential details to other test team members. The topic of test specification optimization is included as a main agenda of these meetings. In addition, the entire system test team meets once a week to discuss impediments.

The approach and progress of the system test effort optimization is communicated to all test teams and also to other stakeholders as part of management reports.

2) Training and applying learning

Detailed guidelines drawn for test specification optimization helped the test engineers to apply them. Test engineers were provided with hands-on training on test design techniques. In addition, hand holding by senior testers, system analysts, and architects also helped. This has opened an unexplored possibility of designing test cases in more structured way. Applying suitable test design techniques for existing and new test cases has reduced clutter of repeated test steps and has resulted in cleaner test cases.

3) Increasing quality focus

As studies from Gartner [7] have shown, "The lack of testing and QA standards, as well as a lack of consistency, often lead to business disruption, which can be costly." Gartner also reports that "testing consumes 25% to 50% of the average application life cycle and often is viewed as adding no business value". Adherence to quality, regulatory and organization quality standards is essential to reduce unproductive effort spent on testing. Recognizing the need, testers were provided the same access to team activities, portals and documentation as other team members. The test manager provided guidelines for addressing quality factors for each test cases adhering to ISO/IEC 25010. This approach provided a suitable 'ility' test with least possible effort.

D. People challenges

1) Evaluation of perception

It is important not only to set a goal and make plan for implementation, but also to monitor the perception/acceptance of test engineers for the entire process. A questionnaire on three parameters was prepared to find out if the testers: 1) were aware of the goals, 2) believed the approach was correct, and 3) were confident to achieve the test effort reduction. Spread over a period of 4 months, the answers to this survey revealed increasingly positive perceptions, which was in line with results achieved.

2) Correction steps

The approach and goals were modified based on the results achieved. The approach to merge similar requirements was replaced by removal of redundant requirements where possible. When the creation of automated test scripts demanded high efforts, the goal to automate all test cases was replaced by partial automation of only the effort intensive manual test steps.

3) Reward and motivation

Efforts of test engineers, who achieved potential reduction estimated and become examples for others, were appreciated

by test resource managers. This also helped improve the interpersonal relationships between manager and test team members. Assigning them roles to coach and mentor other test engineers during the optimization, increased their motivation levels.

E. Results achieved

The resulting reduction in system testing effort is presented in table II. The reduced effort allowed the test team to perform other types of testing to increase test effectiveness, and the increased quality of test specifications and test cases has reduced the effort to review and maintain these test cases.

1) Increase in effort available for other types of testing

With the proportion of system testing effort getting reduced, it was possible for the test team to allocate additional effort for other tests like wild user tests, workflow tests. This contributed to better defect detection as well.

TABLE II. PROPORTION* OF TEST EFFORT BEFORE AND AFTER OPTIMIZATION

Activity	Practice (Section)	Estimated saving of total effort (%)	
Test specification update and reporting	Optimizing multiple test cases per requirement (VI A 1)	6	9
	Same requirement traced to multiple test specifications (VI A 2)	1	
	Repeated test cases in different test configurations (VI A 3)	2	
Effort for test environment preparation	Assisted installation and configuration (VI A 7)	2	2
Formal test execution	Automation (VI A 6)	3	7
	Merging two levels of system testing (VI A 4)	2	
	Reducing time taken for bring-up tests (VI A 8)	1	
	Other methods (VI A 10)	1	
Total effort saved (%)		18	

*Final milestone is not reached yet. The values will increase once the final release milestone is reached

2) The overall reduction efforts for test specification review and maintenance

Not only the test specifications became more concise, the effort to review and maintain them also got reduced.

VII. SUMMARY AND CONCLUSION

The practices helped reduce system test effort by 18 percent at the time of writing this paper. When planned test automation is completed, the effort reduction is expected to be more than 25 percent. The effort reduction was achieved, while working within process guidelines and without requiring large effort from tester and other stake holders. These practices are not generic for the just for our application domain alone and can be applied to any project of similar complexity, by adapting them

to its processes. The rigor in these practices helped manage the expectations of the key stakeholders.

VIII. REFERENCES

- [1] Managing Requirements Volatility while 'Scrumming' within the V Model by Anita PC, Deepti Savio, VS Mani
- [2] Adopting Agile in an FDA, Regulated Environment" [Abbott 2009], Abbot Labs
- [3] A Practitioner's Guide To Software Test Design, Lee Copeland
- [4] Stakeholders in Testing – Tulasi Anand, Siemens Software Initiative – Exchange of Experience – Testing, 2008
- [5] Test effort optimization by prediction and ranking of fault-prone software modules, Pandey, A.K, Goyal, N.K.; Reliability Eng. Centre, Indian Institute of Technology Kharagpur, Kharagpur, India; 2nd International Conference on Reliability, Safety and Hazard (ICRESH), 2010
- [6] Practices to make agile test teams effective: challenges and solutions, Tulasi Anand, V. S. Mani, IEEE ICGSE 2015
- [7] Partha Iyengar, Frances Karamouzis. "Offshore Application Testing Drives Greater Business Value." Gartner Research, August 17, 2007, ID Number G00150394